

Self-RAG (Self-Reflective RAG) — Study Notes

Notes on the limitations of traditional RAG, the Self-RAG architecture and the four questions it answers at each step, and a from-scratch, iterative implementation in LangGraph — covering the retrieval decision, relevance filtering, hallucination checking with a revision loop, and usefulness checking with query rewriting.

1 Three Major Problems with Traditional RAG

1.1 1. Indiscriminate / Unnecessary Retrieval

Traditional RAG always retrieves, even when retrieval is unnecessary. This is a significant flaw.

Example: A children’s chatbot built over encyclopedia books is asked ”How many seconds are there in a minute?” — trivial general knowledge the LLM already knows. But because it’s wired as a RAG chatbot, it still retrieves chunks first (e.g., one about minutes as time units, one about colloquial usage, one about the history of time units). Forced to answer using this unnecessary context, the LLM produces a *less confident* answer (”typically about 60 seconds depending on context”) than it would have unprompted — and computation is wasted on retrieval that added no value.

This is called **indiscriminate retrieval** — retrieving regardless of whether the question actually needs it.

1.2 2. Blind Trust in Retrieved Documents

Traditional RAG blindly trusts whatever is retrieved, even if it doesn’t actually answer the question.

Example: Query: ”What causes diabetes?” The retrieved document (fetched via semantic similarity, since it’s topically about diabetes) describes what diabetes *affects*, not what *causes* it. Forced to answer from this chunk anyway, the LLM produces a logically odd, factually wrong answer — because the document was topically similar but not actually relevant to the specific question asked.

1.3 3. No Verification of Its Own Answers

Once RAG generates an answer, it’s presented as final with no further check on whether it’s actually correct, hallucinated, or grounded in the retrieved evidence.

Self-RAG is designed to address all three of these problems.

2 What Is Self-RAG?

Self-RAG stands for **Self-Reflective RAG**, where the LLM actively judges its own retrieval evidence and answers instead of blindly trusting retrieved documents.

Its core USP is **self-reflection**: at every step, the architecture judges its own actions — asking whether what it just did was actually correct — and adjusts accordingly.

2.1 The Four Key Questions Self-RAG Answers

1. **Is retrieval needed at all** for this query, or can the model answer directly from its parametric knowledge?
2. **Are the retrieved documents relevant** to answering the question?
3. **Is the generated response grounded** in the retrieved documents — i.e., does every fact in the answer actually come from the retrieved evidence, or did the model fabricate/hallucinate some of it?
4. **Does the response actually answer the user's question** — even if every fact is correct and grounded, does it actually address what was asked?

Example (Q3 — groundedness): Query: "What are the side effects of Drug X?" Retrieved docs mention it's prescribed for hypertension, and that trials reported dizziness and nausea. The LLM answers: "Drug X may cause dizziness, nausea, fatigue and headaches, especially in older patients." Dizziness and nausea come from the documents — but fatigue, headaches, and the "older patients" claim were **fabricated** from the model's own parametric knowledge of similar drugs. This is a form of hallucination Self-RAG is designed to catch.

Example (Q4 — usefulness): Query: "Why does ice float on water?" Retrieved doc: "Ice is the solid form of water." The LLM answers: "Ice is the solid form of water that occurs at low temperature" — grounded in the document, but it **doesn't actually answer why ice floats**. Self-RAG catches this even though there's no hallucination.

3 Self-RAG Architecture: Conceptual Flow

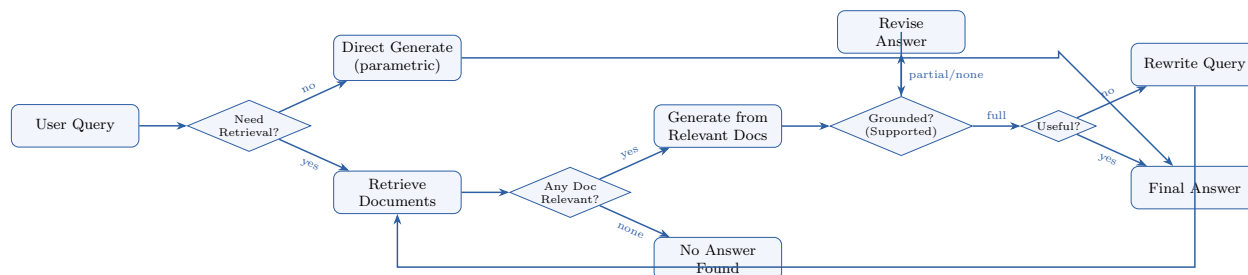


Figure 1: Self-RAG conceptual flow: retrieval decision, relevance filtering, groundedness check with a revise loop, and usefulness check with a rewrite-and-retry loop.

3.1 Step-by-Step Walkthrough

1. **Retrieval decision:** the query is examined; if it can be answered from general/parametric knowledge (e.g., "What is a paid leave?"), the LLM answers *directly* and the flow ends. If it needs specific facts (e.g., "How many paid leave days do employees get per year?"), retrieval proceeds.
2. **Relevance filtering:** each retrieved document is individually judged relevant or not to the query. Only relevant documents survive; if **none** are relevant, the flow immediately prints "no answer found" and terminates.
3. **Generation:** if at least one document is relevant, the answer is generated from the relevant document(s).
4. **Groundedness check (hallucination detection):** the generated answer is classified into one of three categories:
 - **Fully supported** — every fact in the answer traces back to the retrieved documents.
 - **Partially supported** — some facts come from the documents, but the model fabricated additional claims/correlations not actually stated (e.g., inferring that sick and casual leave count *within* the total paid leave figure, when the documents never say this).
 - **No support** — the answer is essentially entirely fabricated, with no grounding in the retrieved documents at all (proper hallucination).

If fully supported, the answer proceeds forward. If partially supported or no support, it's sent to a **revise answer** step, which rewrites the answer using a strict prompt to remove any fact not present in the context — and the revised answer loops back to be re-checked. A max-retries counter (e.g., 5 or 10) prevents an infinite loop.

5. **Usefulness check:** once an answer is fully supported (non-hallucinating), it's still checked for whether it actually *answers* the user's question. If useful, the flow ends and the answer is shown to the user. If not useful, the **original query is rewritten** and the entire flow (retrieve → filter → generate → check groundedness → check usefulness) repeats from the top with the new query — again bounded by a max-retries counter, falling back to "no answer found" if exceeded.

This directly maps onto the four questions: retrieval decision answers Q1, relevance filtering answers Q2, the groundedness/support check answers Q3, and the usefulness check answers Q4.

Implementation note: the original Self-RAG paper used a specially fine-tuned model for these judgment steps. The implementation here substitutes general-purpose OpenAI LLMs with structured outputs instead, since the fine-tuned model isn't publicly available — the core ideas are the same, but implementation details differ from the paper.

4 Building Self-RAG Iteratively in LangGraph

The system is built incrementally in LangGraph, one feature at a time, using a hypothetical company ("Nexa AI Solutions") with documents on company profile, policies, and product/pricing as the RAG corpus.

4.1 Step 1: The Retrieval Decision Node

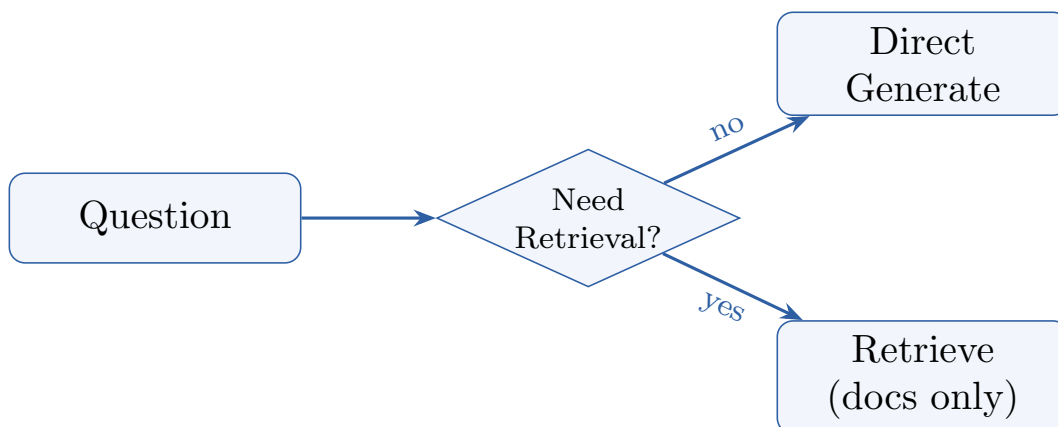


Figure 2: Iteration 1: decide whether retrieval is needed; retrieve documents but don't generate from them yet.

- **State:** question, `need_retrieval` (boolean), `docs` (retrieved documents), final answer.
- **Decide-retrieval node:** a Pydantic schema forces a true/false `should_retrieve` output. System prompt: return true if answering reliably requires specific facts/citations likely not in the model's own knowledge; false for general explanations, definitions, or reasoning; when unsure, default to true.
- **Direct-generate node:** answers using only general knowledge, explicitly instructed not to assume access to external documents, and to say "I don't know based on my general knowledge" if unsure.
- **Retrieve node:** simply invokes the retriever with the question.
- **Routing function:** routes to `retrieve` if `need_retrieval` is true, else to `direct_generate`.

Testing: "Who is the CEO of Nexa AI?" → `need_retrieval` = `true`, documents retrieved (no answer generated yet, since generation-from-retrieval isn't built yet). "What is machine learning?" → `need_retrieval` = `false`, answered directly, no documents retrieved.

4.2 Step 2: Filtering Relevant Documents

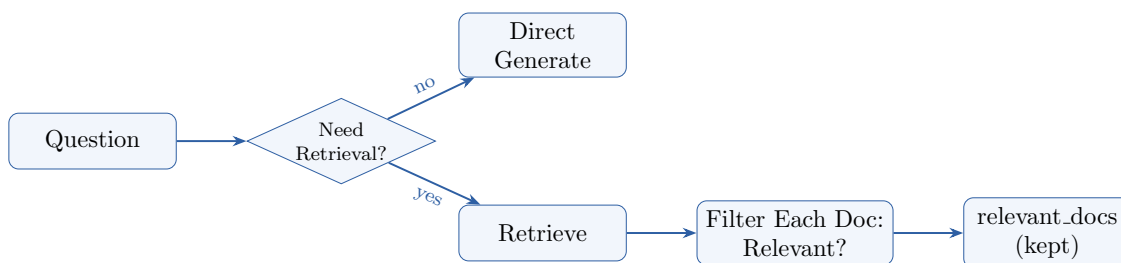


Figure 3: Iteration 2: each retrieved document is individually judged relevant or not; only relevant ones are kept.

- New state key: `relevant.docs` (subset of `docs`).

- **Is-relevant node:** a Pydantic schema with a single boolean field; system prompt judges whether a document contains information useful for answering the question. Looped over every retrieved document; kept ones are appended to `relevant_docs`.

Testing "Who is the CEO of Nexa AI?": of 4 retrieved documents, only 1 (the one actually discussing the CEO) was kept in `relevant_docs` — the rest were semantically close but not actually relevant, and correctly filtered out.

4.3 Step 3: Generating Answers from Context

- New state key: `context` (string) — all `relevant_docs` merged together.
- **Generate-from-context node:** prompt instructs the LLM to act as a "business RAG assistant" and answer using *only* the provided context.
- **No-relevant-docs node:** a placeholder node returning "no answer found" when zero documents are relevant — later replaceable with a web-search fallback (an alternative path is briefly sketched: if no document is relevant, rewrite the query for web search, fetch results, and re-check relevance — though this fallback isn't built into the final Self-RAG implementation here).
- **Routing:** if `len(relevant_docs) > 0`, go to generate-from-context; else go to no-relevant-docs.

Testing: "Who is the CEO of Nexa AI?" → correct answer from the relevant document. "What is the refund policy of Nexa AI?" (not covered in the docs) → "no relevant document found."

4.4 Step 4: Detecting Hallucinations (the Support Node)

- New state keys: `support` (fully_supported / partially_supported / no_support) and `evidence` (a list of facts extracted from context, mainly for debugging/traceability).
- **Is-supported node:** a Pydantic schema captures the support verdict and the evidence list. The LLM is given the question, the generated answer, and the context, and judges whether every claim in the answer is grounded in the context.

Test 1: "How many employees does Nexa AI have?" → `fully_supported` — the answer ("85+ employees") and its evidence trace directly to the context.

Test 2: "Describe Nexa AI company culture" → `partially_supported` — the LLM added some claims beyond what the retrieved evidence actually stated.

Test 3: "Does Nexa AI have a free trial? If so, how many days?" (not in the documents) → `no_support` — with empty evidence, the LLM fell back on parametric knowledge (a plausible-sounding "14 days," a common industry pattern) despite having no actual grounding — correctly caught as hallucination.

4.5 Step 5: The Revise-Answer Loop

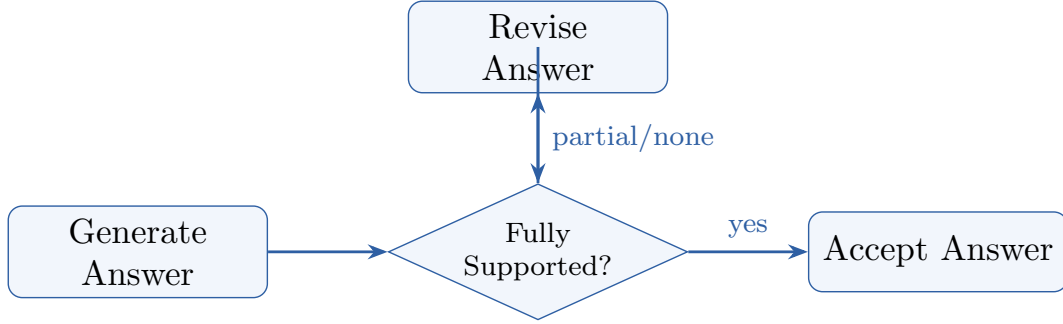


Figure 4: The groundedness loop: partially- or non-supported answers are revised and re-checked, bounded by a max-retries counter.

- New state key: **retries** (counts loop iterations; a max-retries limit, e.g. 5, breaks the loop to prevent infinite looping).
- **Revise-answer node:** a strict prompt instructs the LLM to rewrite the current answer so that it is based *only* on the given context, given the question, current answer, and context; produces a **revised_answer**, and increments **retries**.
- The revised answer is routed back into the **is-supported** check, forming a loop that continues until the answer is fully supported (or retries are exhausted).

Testing: a query previously judged "partially supported" was re-run; after one revision cycle, the support verdict became "fully supported," with the answer trimmed down to only what the evidence actually stated.

4.6 Step 6: Testing Usefulness and Query Rewriting

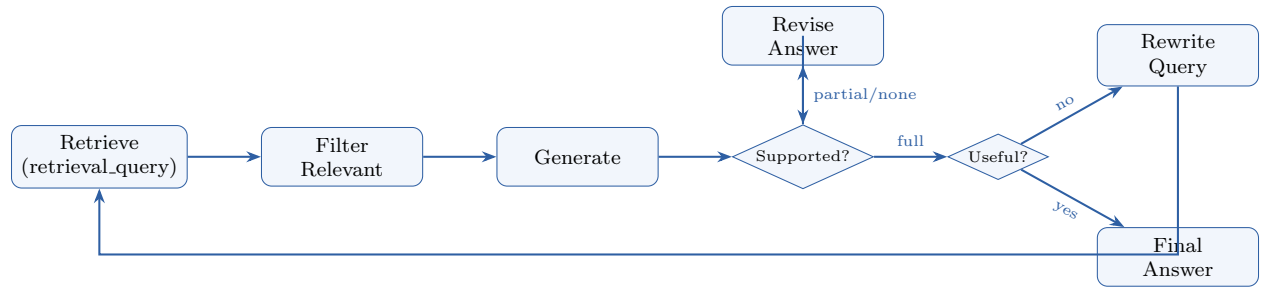


Figure 5: Complete Self-RAG loop: usefulness failures trigger a query rewrite and a full re-run of retrieve → filter → generate → groundedness check → usefulness check, bounded by a max-retries counter.

- New state keys: **is_useful** (useful / not_useful, with a reason) and, for the final version, **retrieval_query** (initially equal to the original question, but rewritten on retry) and **rewrite_tries**.
- **Is-useful node:** given the question and the (fully supported) generated answer, the LLM judges whether the answer actually justifies/answers the question. Routes to **finalize** (end) if useful, or to a "no answer found" placeholder if not — this placeholder is later replaced with the rewrite loop.

- **Rewrite-query node:** given the original question, the previous retrieval query, and the current answer, an LLM rewrites the question into a query optimized for vector retrieval against the internal documents — kept short, preserving key entities, adding 2–5 high-signal keywords. This becomes the new `retrieval_query`.
- **Retrieve node update:** now retrieves using `retrieval_query` instead of the raw original question, so it reflects any rewrites from prior loop iterations.
- The full pipeline (retrieve → filter → generate → check support/revise → check usefulness) repeats using the rewritten query, bounded by a max-retries counter that falls back to "no answer found" if exceeded.

Testing: "Who is the CEO of Nexa AI?" → fully supported, useful, correct final answer. "What is the refund policy of Nexa AI?" (not in documents) → no support, not useful, correctly resolves to "no answer found" after exhausting retries. "Describe Nexa AI company culture" → after one revision retry, reaches fully supported *and* useful, producing a grounded final answer.

5 Quick Revision Summary

1. Traditional RAG has three core problems: it retrieves indiscriminately even when unnecessary, blindly trusts retrieved documents regardless of true relevance, and never verifies its own generated answers.
2. Self-RAG's core idea is **self-reflection**: at every step the system judges its own actions rather than proceeding blindly.
3. It answers four questions: (1) is retrieval needed, (2) are the retrieved documents relevant, (3) is the generated answer grounded in the documents (fully/partially/not supported), and (4) does the answer actually address the user's question (useful or not).
4. If retrieval isn't needed, the LLM answers directly from parametric knowledge; if no retrieved document is relevant, the flow terminates with "no answer found."
5. Groundedness is checked via a **support** classification (fully/partially/no support); partially- or non-supported answers are sent through a **revise-answer** loop (bounded by max retries) until fully supported.
6. Usefulness is checked separately from groundedness — a fully grounded answer can still fail to actually answer the question. Unuseful answers trigger a **query rewrite** and a full re-run of the retrieve-through-generate pipeline (also bounded by max retries).
7. The implementation substitutes general-purpose LLMs with structured outputs for the paper's original fine-tuned judgment model, since that model isn't publicly available — the architecture's core logic remains faithful to the original design.